

■■ Hoja de Referencia Rápida: H2 Database con Spring Boot, JPA & Maven

■ 1. Introducción y Modos de Operación

H2 Database es un motor de base de datos relacional escrito en Java, de alto rendimiento y footprint reducido. Es la solución estándar en Spring Boot para entornos de desarrollo rápido, pruebas unitarias e integración, y laboratorios/evaluaciones.

Modos de Ejecución:

- **En Memoria (in-memory):** La base de datos reside únicamente en RAM (`jdbc:h2:mem:nombredb`). Los datos se destruyen al reiniciar la aplicación.
- **Basado en Archivo (file-based):** Persiste los datos en archivos físicos del disco local (`jdbc:h2:file:./data/nombredb`).
- **Modo Servidor (TCP):** Permite conexiones simultáneas remotas mediante socket de red TCP.

■ 2. Dependencia Maven (pom.xml)

Para utilizar H2 Database en un proyecto Spring Boot con Spring Data JPA, agregue el siguiente bloque de dependencia al archivo `pom.xml`:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

■ ■ 3. Propiedades de Configuración

Configure las credenciales y el comportamiento del motor de base de datos en `src/main/resources/application.properties` o `application.yml`.

A. Configuración en `application.properties`:

```
# Conexion JDBC H2 en Memoria
spring.datasource.url=jdbc:h2:mem:educonnectdb;\
DB_CLOSE_DELAY=-1
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Dialecto Hibernate para H2
spring.jpa.database-platform=\
org.hibernate.dialect.H2Dialect

# Desactivar Auto-DDL de Hibernate
# para priorizar schema.sql
spring.jpa.hibernate.ddl-auto=none

# H2 Console Web
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# Ejecucion automatica de scripts SQL
spring.sql.init.mode=always

# Mostrar SQL en consola
spring.jpa.show-sql=true
```

B. Configuración Equivalente en `application.yml`:

```
spring:
  datasource:
    url: jdbc:h2:mem:educonnectdb;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    database-platform: org.hibernate.dialect.H2Dialect
```

```
hibernate:
  ddl-auto: none
  show-sql: true
h2:
  console:
    enabled: true
    path: /h2-console
sql:
  init:
    mode: always
```

■ 4. Scripts de Inicialización ANSI SQL (`schema.sql` y `data.sql`)

Spring Boot ejecuta de forma automática los archivos ubicados en `src/main/resources/`: 1. `schema.sql`: Sentencias DDL para creación de tablas y restricciones. 2. `data.sql`: Sentencias DML INSERT INTO para la carga inicial de datos.

Ejemplo de `schema.sql` (DDL):

```
DROP TABLE IF EXISTS matricula_curso;
DROP TABLE IF EXISTS estudiante;
DROP TABLE IF EXISTS profesor;

CREATE TABLE profesor (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  especialidad VARCHAR(80) NOT NULL,
  activo BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE estudiante (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  carnet VARCHAR(20) NOT NULL UNIQUE,
  nombre_completo VARCHAR(120) NOT NULL,
  correo VARCHAR(100) NOT NULL
);

CREATE TABLE matricula_curso (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  codigo_matricula VARCHAR(30) NOT NULL UNIQUE,
```

```

    estudiante_id BIGINT NOT NULL,
    profesor_id BIGINT NOT NULL,
    nombre_curso VARCHAR(100) NOT NULL,
    creditos INT NOT NULL,
    estado VARCHAR(20) NOT NULL,
    fecha_matricula TIMESTAMP NOT NULL,
    CONSTRAINT fk_mat_est
        FOREIGN KEY (estudiante_id)
        REFERENCES estudiante(id),
    CONSTRAINT fk_mat_prof
        FOREIGN KEY (profesor_id)
        REFERENCES profesor(id)
);

```

Ejemplo de `data.sql` (DML):

```

INSERT INTO profesor
(nombre, especialidad, activo)
VALUES
('Dr. Armando Ramos', 'Computacion', TRUE),
('Dra. Beatriz Solis', 'Sistemas', TRUE);

INSERT INTO estudiante
(carnet, nombre_completo, correo)
VALUES
('C01234', 'Valeria Monge', 'valeria@ucr.ac.cr'),
('C05678', 'Kevin Alvarado', 'kevin@ucr.ac.cr');

INSERT INTO matricula_curso
(codigo_matricula, estudiante_id, profesor_id,
nombre_curso, creditos, estado, fecha_matricula)
VALUES
('MAT-2026-001', 1, 1, 'DSW IV', 4,
'ACTIVA', CURRENT_TIMESTAMP);

```

■ 5. Mapeo JPA Compatible con H2

Cree sus entidades anotadas en Java. H2 asigna `BIGINT AUTO_INCREMENT` mediante la estrategia `GenerationType.IDENTITY`:

```

package com.educonnect.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "matricula_curso")
public class MatriculaCurso {

    @Id
    @GeneratedValue(
        strategy = GenerationType.IDENTITY
    )
    private Long id;

    @Column(
        name = "codigo_matricula",
        nullable = false,
        unique = true
    )
    private String codigoMatricula;

    @ManyToOne(optional = false)
    @JoinColumn(
        name = "estudiante_id",
        nullable = false
    )
    private Estudiante estudiante;

    @ManyToOne(optional = false)
    @JoinColumn(
        name = "profesor_id",
        nullable = false
    )
    private Profesor profesor;

    @Column(nullable = false)
    private String estado;

    @Column(
        name = "fecha_matricula",
        nullable = false
    )
    private LocalDateTime fechaMatricula;

    // Getters y Setters

```

```
}
```

■ 6. Acceso y Uso de la Consola Web (H2 Console)

Spring Boot incluye una interfaz gráfica basada en navegador para administrar H2.

1. **URL de Acceso:** `http://localhost:8080/h2-console`
2. **Campos del Formulario de Ingreso:**
3. **Driver Class:** `org.h2.Driver`
4. **JDBC URL:** Deberá coincidir **exactamente** con la propiedad declarada en `application.properties` (ej: `jdbc:h2:mem:educonnectdb`).
5. **User Name:** `sa`
6. **Password:** (Dejar en blanco)

[!CAUTION] **Causa de Error Común:** Si al ingresar a la consola no visualiza las tablas creadas, verifique que el valor de **JDBC URL** en la pantalla de login de H2 sea idéntico al de `spring.datasource.url`. De lo contrario, H2 creará una base de datos alternativa completamente vacía.

■ 7. Pruebas de Persistencia con `@DataJpaTest`

H2 Database se integra de forma transparente con Spring Boot Test para la ejecución de pruebas unitarias y de integración de la capa de datos:

```
package com.educonnect.repository;  
  
import com.educonnect.model.Profesor;  
import org.junit.jupiter.api.Test;  
import org.springframework.beans.factory  
    .annotation.Autowired;  
import org.springframework.boot.test.autoconfigure
```

```

    .orm.jpa.DataJpaTest;

import java.util.Optional;

import static org.junit.jupiter.api
    .Assertions.*;

@DataJpaTest
class ProfesorRepositoryTest {

    @Autowired
    private ProfesorRepository repository;

    @Test
    void guardarProfesor_Exito() {
        Profesor prof = new Profesor();
        prof.setNombre("Dr. Ramon");
        prof.setEspecialidad("IA");
        prof.setActivo(true);

        Profesor guardado = repository.save(prof);

        assertNotNull(guardado.getId());
        assertEquals("Dr. Ramon", guardado.getNombre());
    }
}

```

■ 8. Guía de Solución de Problemas (Troubleshooting)

Sintoma / Error	Causa Raíz	Solución Recomendada
Table "XYZ" already exists	Hibernate ddl-auto creando tablas en paralelo a schema.sql.	Ajustar spring.jpa.hibernate.ddl-auto=none en application.properties.

Database is closed al ejecutar queries	La base de datos en memoria se cierra al desconectar la última conexión.	Añadir ;DB_CLOSE_DELAY=-1 al final de la spring.datasource.url.
Table "XYZ" not found en H2 Console	El navegador se conectó a una URL de JDBC por defecto diferida (jdbc:h2:mem:testdb).	Copiar la URL exacta declarada en el backend dentro del campo JDBC URL del login de H2.
Script schema.sql failed al iniciar	Error sintáctico ANSI SQL en el script o discrepancia en tipos.	Validar que se use BIGINT AUTO_INCREMENT y comas correctas en sentencias CREATE TABLE.
